Chapter 8: Transforming Chemistry with Large Language Models – From Chemical to Protein Language Models

Chem 361

Xuhui Huang

Outline

- **Introduction to Large Language Models (LLMs)**
- Transformer
- LLMs in Chemistry and Biology
- Applying a pre-trained Protein LLM to predict GPCR agonists – Hands-on implementation for a real chemistry problem.

© Xuhui Huang

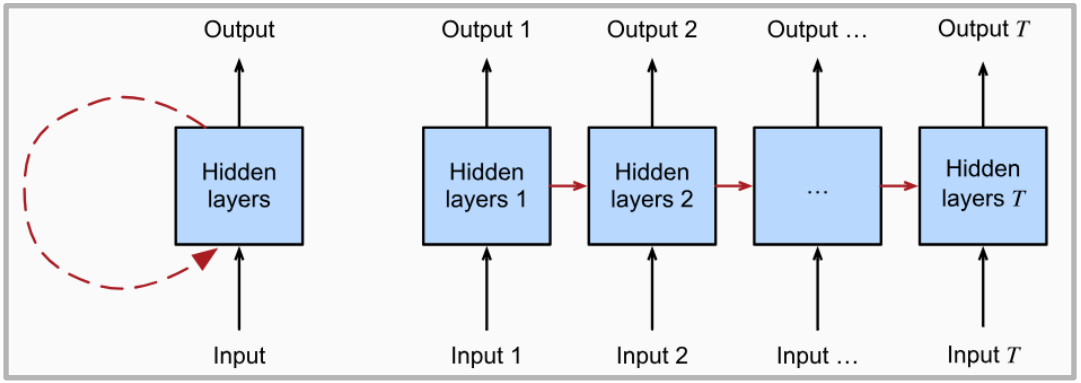
Language Models for Sequential Data

Language:

Previously, when dealing with individual inputs, we assumed that they were sampled independently from the same underlying distribution $P(X)$. While we still assume that entire sequences (e.g., entire documents or patient trajectories) are sampled independently, we cannot assume that the data arriving at each time step are independent of each other. For example, the words that likely to appear later in a document depend heavily on words occurring earlier in the document. The medicine a patient is likely to receive on the 10th day of a hospital visit depends heavily on what transpired in the previous nine days.

- Data type: **Sequential data**

Recurrent neural networks (RNNs)

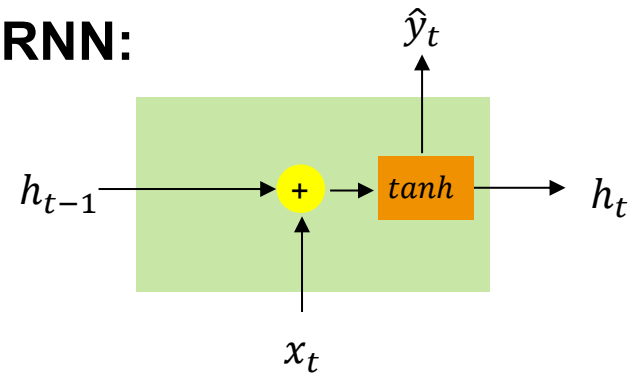


https://d2l.ai/

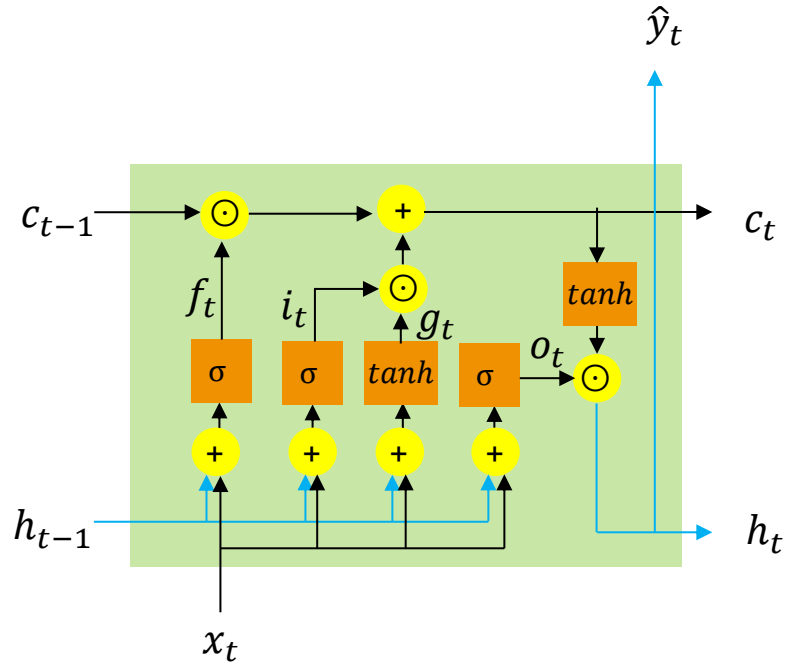
© Xuhui Huang

RNNs as Language Models

RNN:



LSTM:



Learn only long-term memory:

Backpropagation through time learns long-term dependencies of weights.

LSTM extends standard RNNs by replacing each recurrent node with a memory cell. This cell has built-in memory that allows gradients to flow across many time steps without vanishing or exploding.

LSTM learns **both long- and short-term memory** via the regulations of input, forget and output gates.

© Xuhui Huang

Limitations of RNNs

RNN or LSTM:

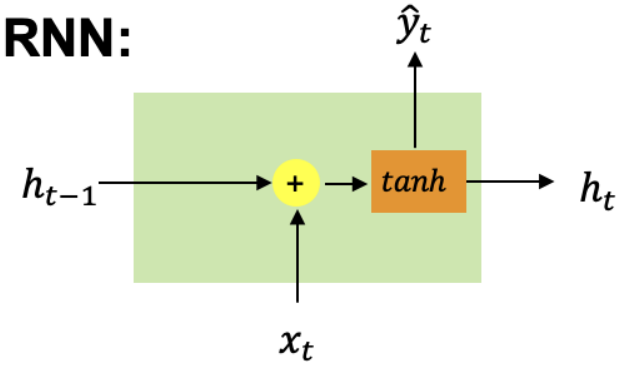
- Sequential processing and limited parallelization.
- Difficulty in handling long-range dependencies.

Example:

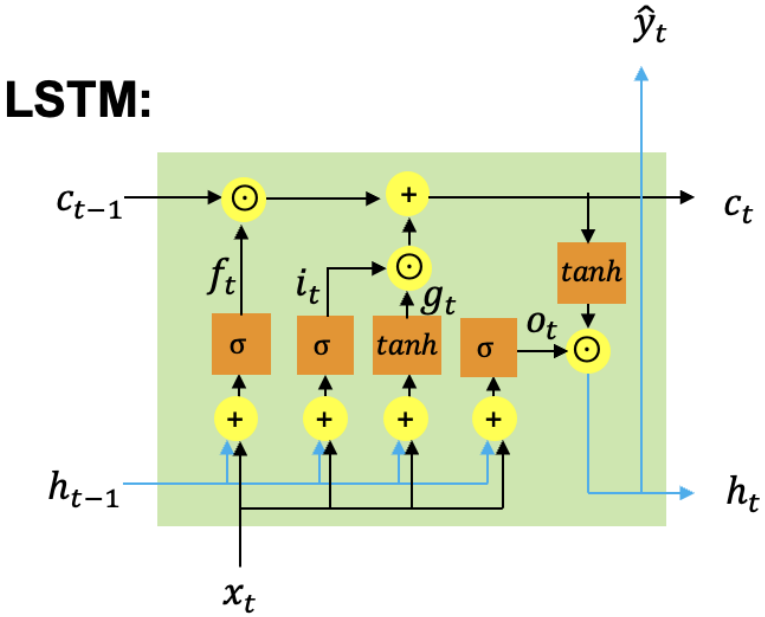
"The book that I borrowed from the library last month turned out to be an interesting novel."

- Gradient vanishing and exploding
- Difficult to scale up

RNN:

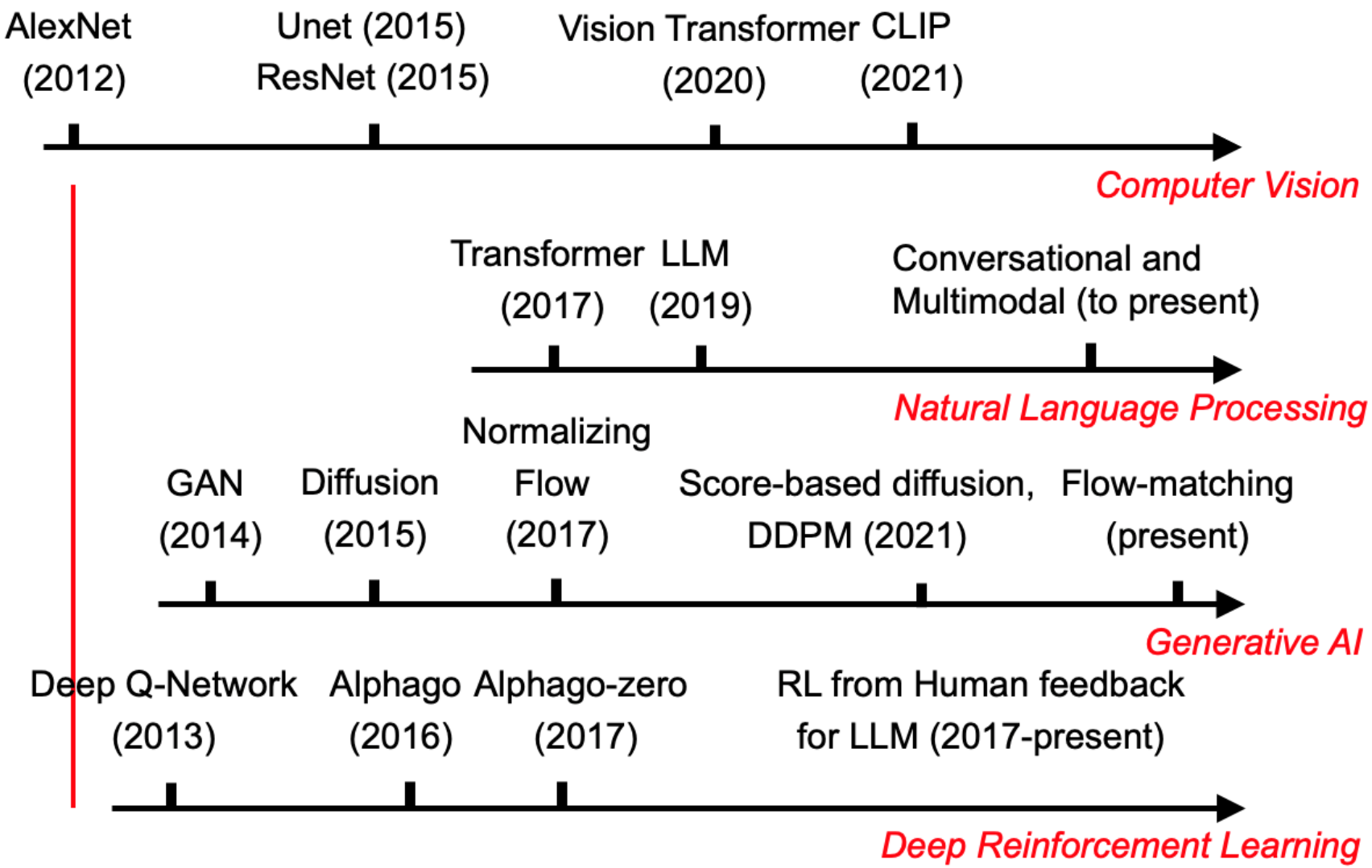


LSTM:



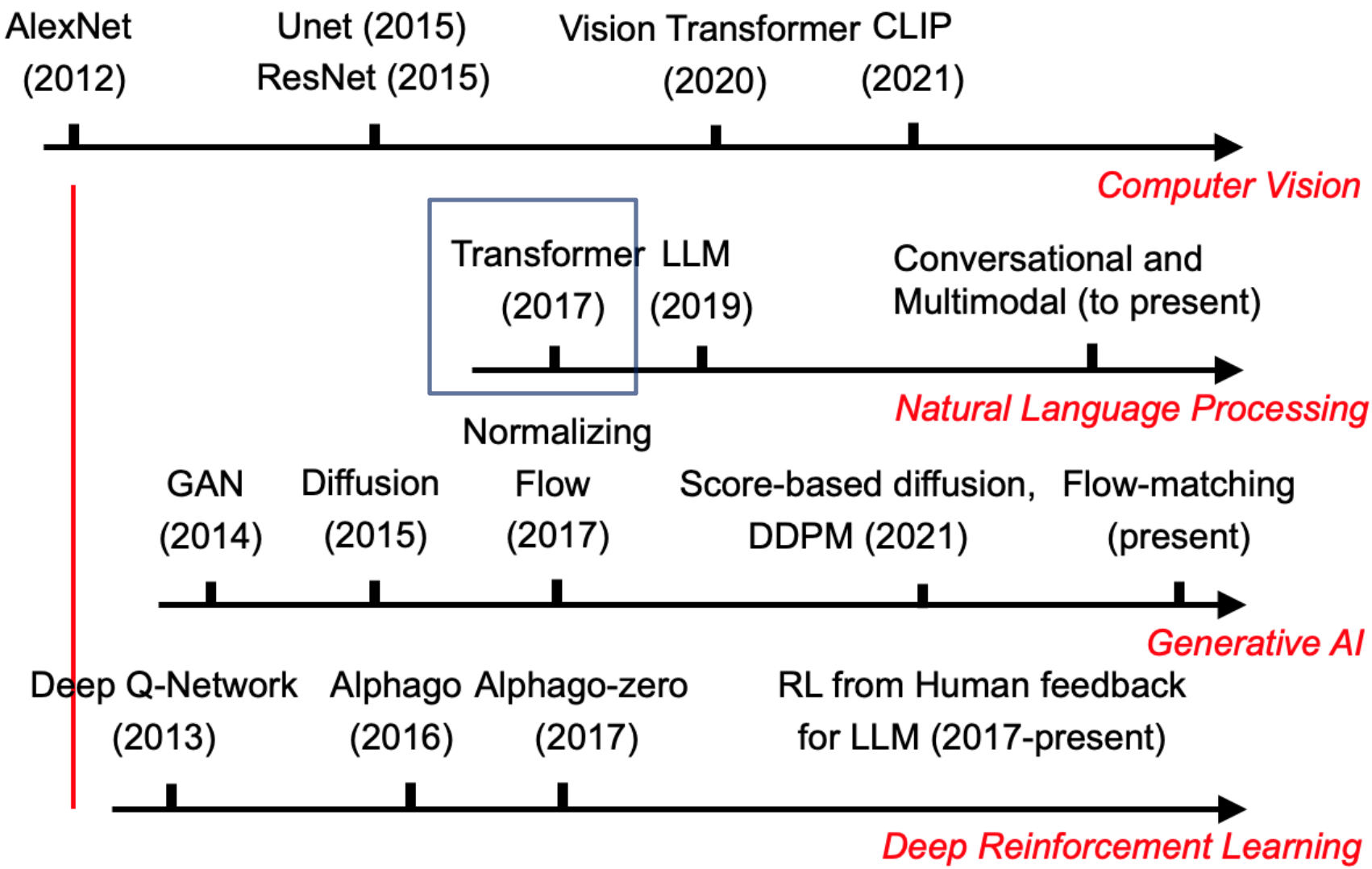
© Xuhui Huang

Timeline of Deep Learning Developments



© Xuhui Huang

Timeline of Deep Learning Developments



© Xuhui Huang

Transformer: A Breakthrough in Language Models

Attention Is All You Need

Ashish Vaswani* Google Brain avaswani@google.com

Noam Shazeer* Google Brain noam@google.com

Niki Parmar* Google Research nikip@google.com

Jakob Uszkoreit* Google Research usz@google.com

Llion Jones* Google Research llion@google.com

Aidan N. Gomez* † University of Toronto aidan@cs.toronto.edu

Łukasz Kaiser* Google Brain lukaszkaiser@google.com

Illia Polosukhin* ‡ illia.polosukhin@gmail.com

(2017)

© Xuhui Huang

Large Language Models (LLMs)

BERT

(2018)

BERT: Pre-training of Deep Bidirectional Transformers for Language Understanding

Jacob Devlin **Ming-Wei Chang** **Kenton Lee** **Kristina Toutanova**
Google AI Language
{jacobdevlin,mingweichang,kentonl,kristout}@google.com

Wikipedia

© Xuhui Huang

Large Language Models (LLMs)

deepseek

© Xuhui Huang

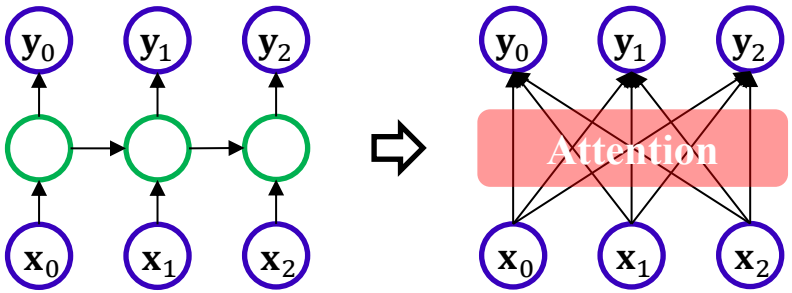
Outline

- Introduction to Large Language Models (LLMs)
- **Transformer**
- LLMs in Chemistry and Biology
- Applying a pre-trained Protein LLM to predict GPCR agonists – Hands-on implementation for a real chemistry problem.

© Xuhui Huang

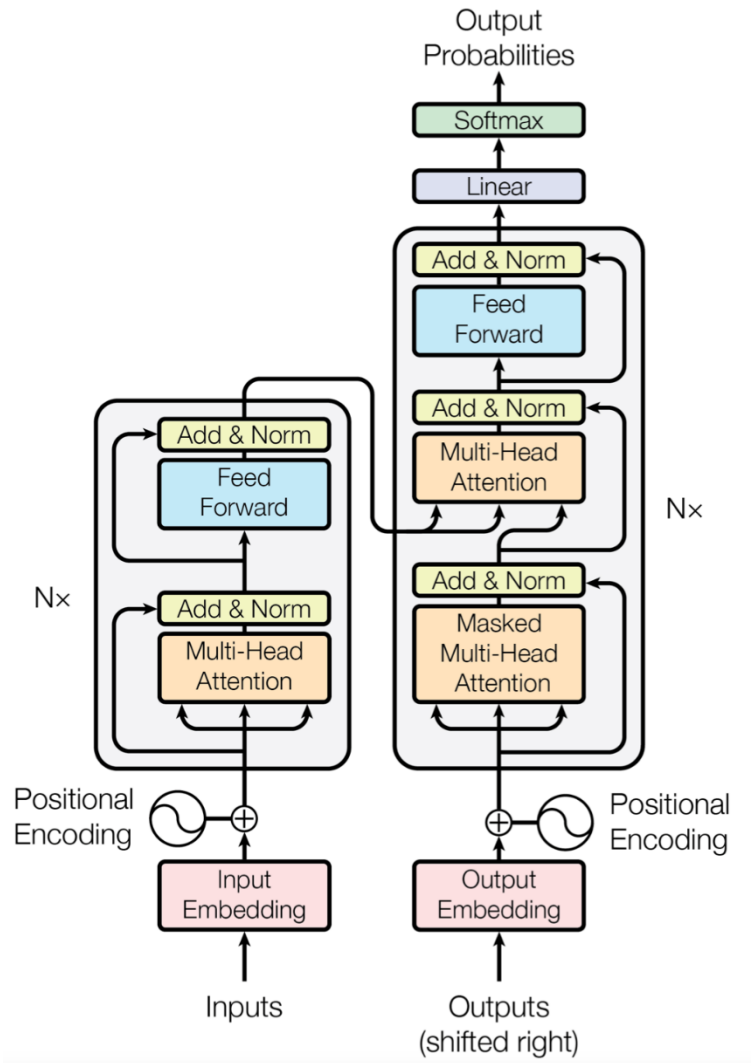
Key Advantages of Transformer

Attention is All You Need!



Recurrence ⟹ Attention mechanism

- Parallelization
- Capture long-range interaction
- Easy to train and scale up

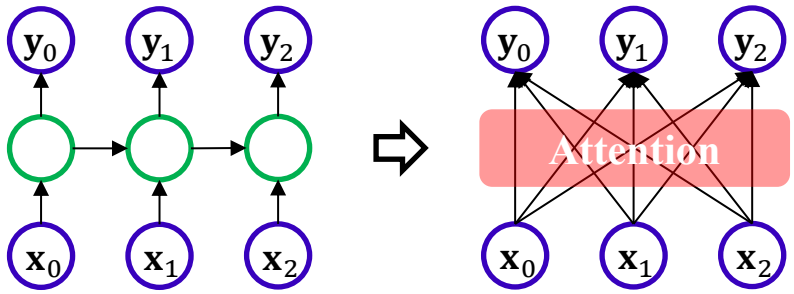


Vaswani, Ashish, et al. "Attention is all you need." *Advances in neural information processing systems* 30 (2017).

© Xuhui Huang

Key Advantages of Transformer

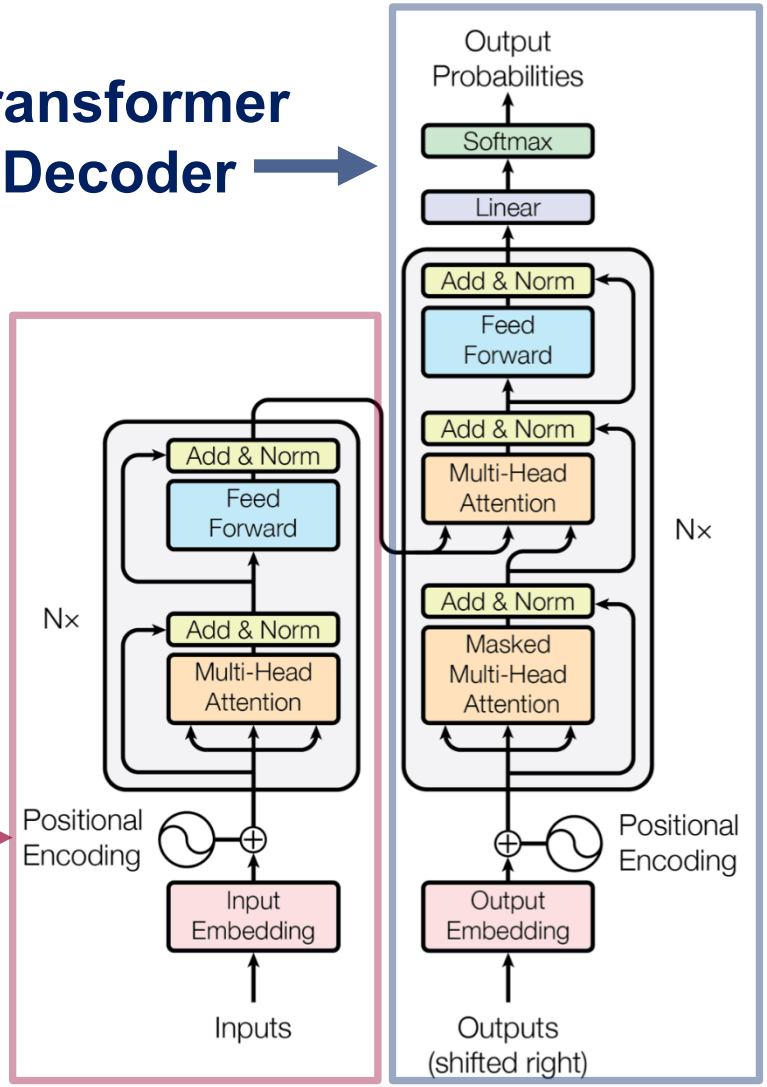
Attention is All You Need!



Recurrence ⟹ Attention mechanism

Transformer Decoder

Transformer Encoder



Vaswani, Ashish, et al. "Attention is all you need." *Advances in neural information processing systems* 30 (2017).

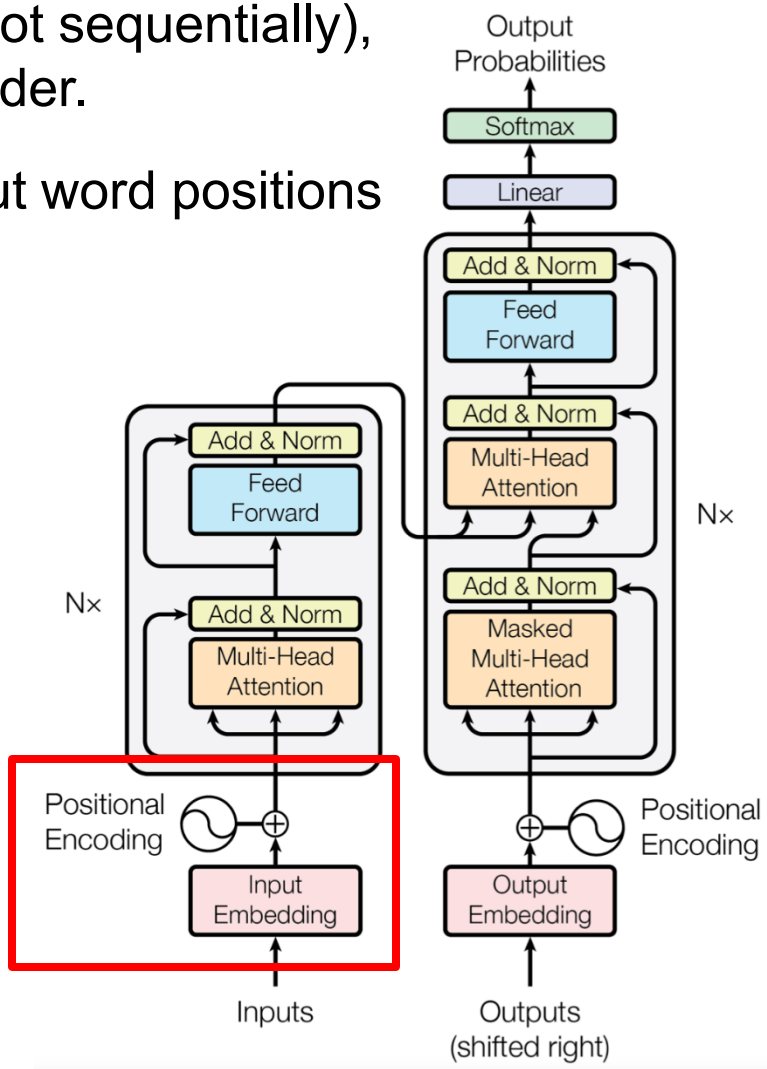
© Xuhui Huang

Positional Encoding

Transformers process all words at once (not sequentially), so they need a way to understand word order.

Position encoding adds information about word positions to the input embeddings.

- A unique position vector is added to each word's embedding.
- These position vectors help the model distinguish where each word is in the sequence.



Vaswani, Ashish, et al. "Attention is all you need." *Advances in neural information processing systems* 30 (2017).

© Xuhui Huang

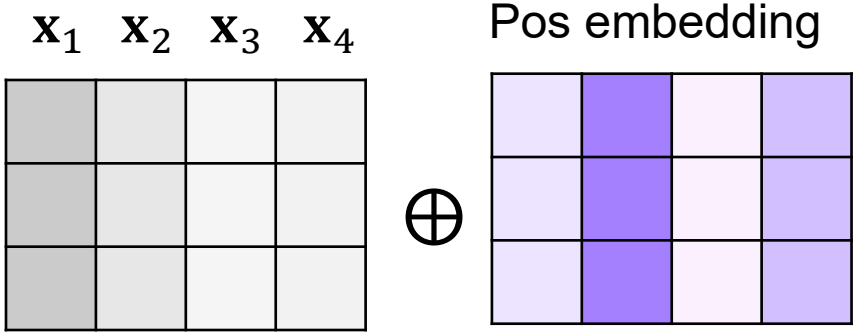
Positional Encoding

Position encoding adds information about word positions to the input embeddings.

Fixed (Sinusoidal): Uses sine and cosine functions to create repeating patterns that represent positions.

- Even Positions: Use the sine function.

- Odd Positions: Use the cosine function.



Learned: Learns position information during training like regular embeddings.

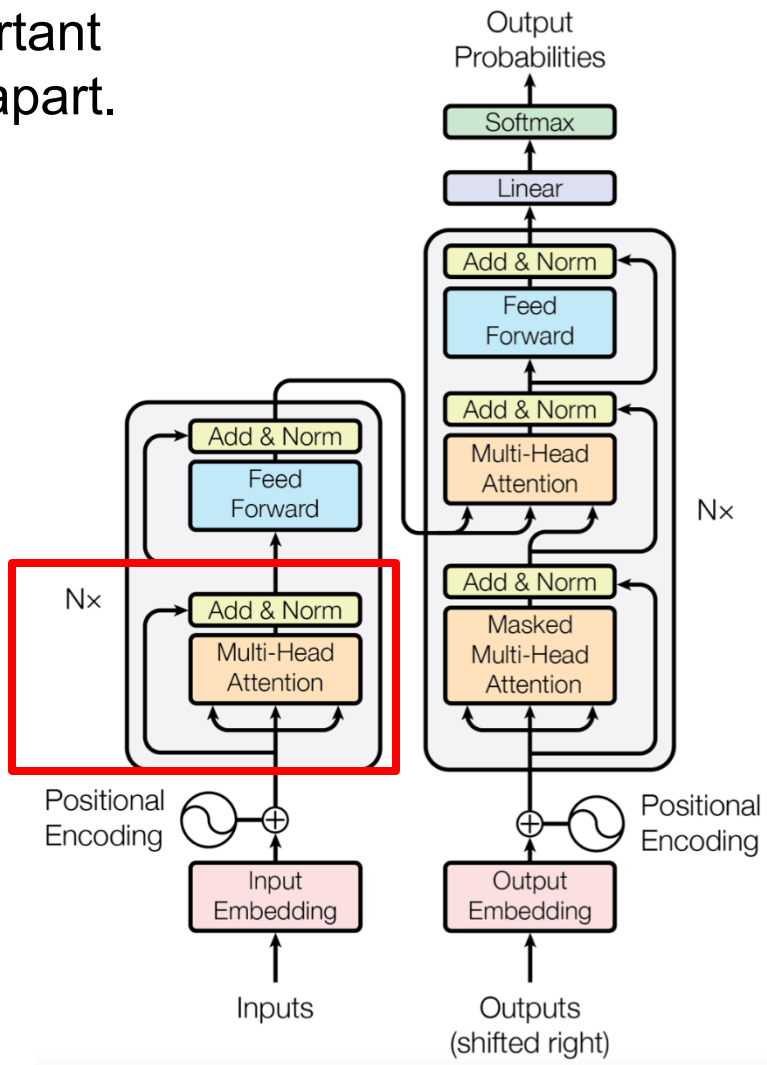
© Xuhui Huang

Attention Mechanism

Attention helps the model focus on important words in a sentence, even if they are far apart.

Self-Attention: Each word compares itself to all other words to determine their importance.

- **Query (Q):** The word that is "asking" for information.
- **Key (K):** The words that are "offering" information.
- **Value (V):** The actual content of the words.



Vaswani, Ashish, et al. "Attention is all you need." *Advances in neural information processing systems* 30 (2017).

© Xuhui Huang

Attention Mechanism

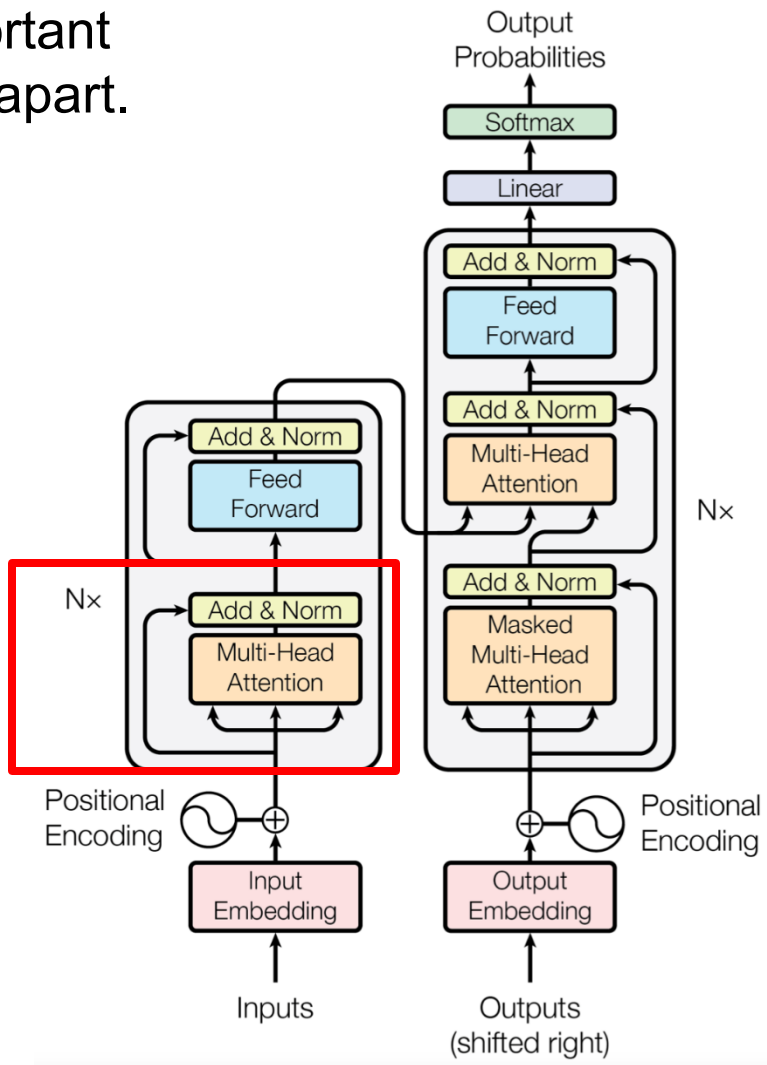
Attention helps the model focus on important words in a sentence, even if they are far apart.

Example: A YouTube video search:

Query (Q): *"Find videos about machine learning basics."* (What the user is looking for)

Keys (K): Titles and descriptions of all available videos. (Potential matches)

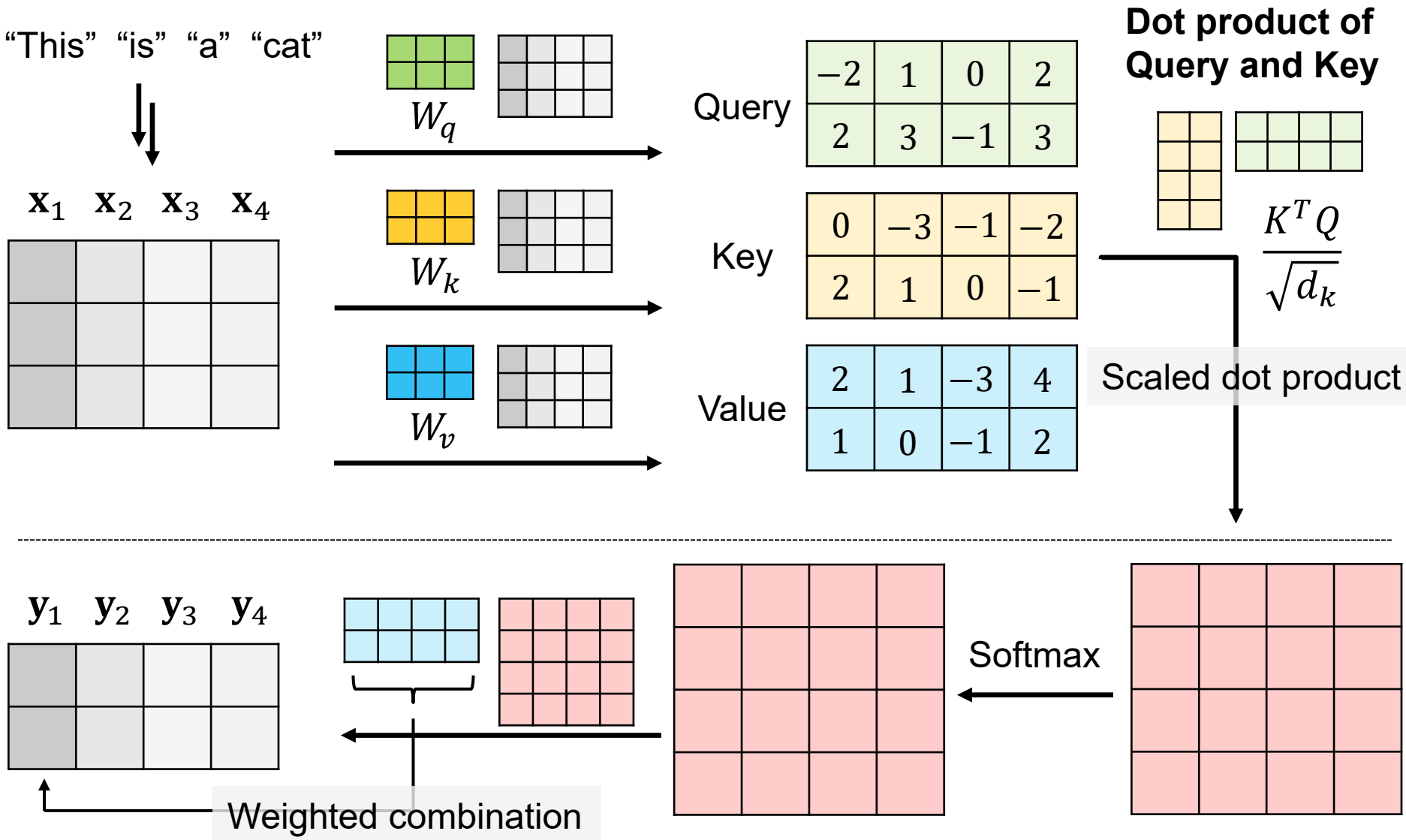
Values (V): Actual video content and metadata. (What the user will get if they click)



Vaswani, Ashish, et al. "Attention is all you need." *Advances in neural information processing systems* 30 (2017).

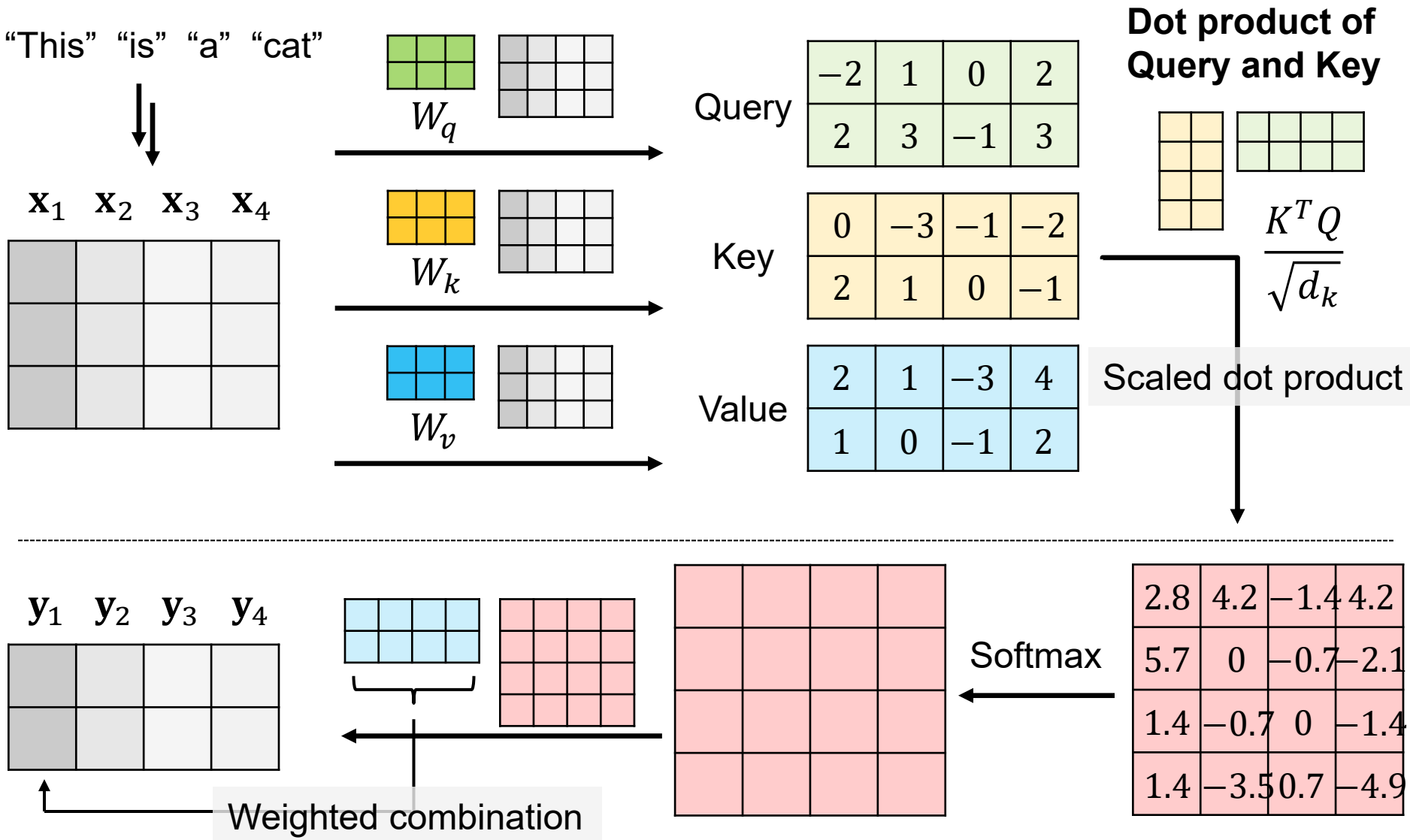
© Xuhui Huang

Attention Mechanism



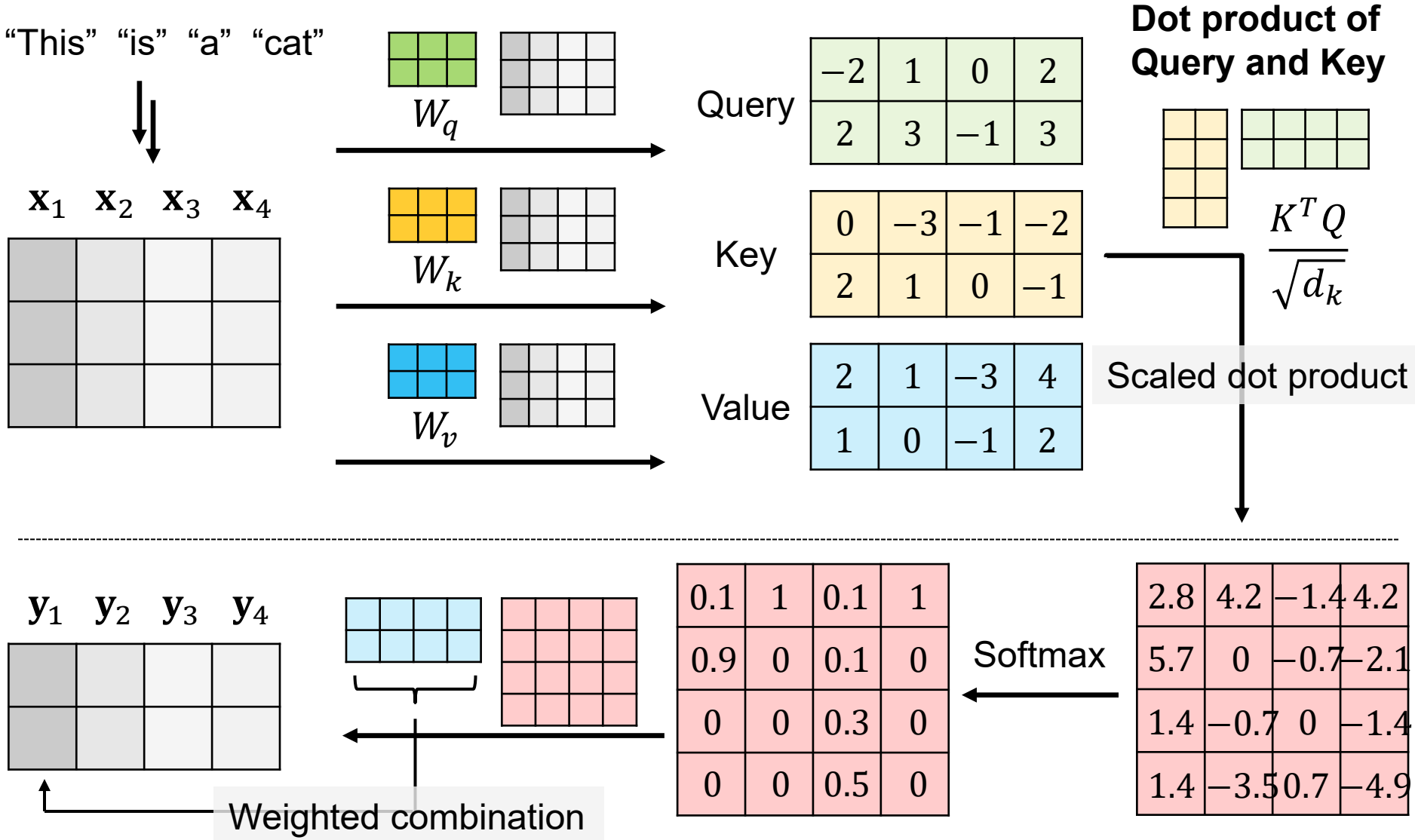
© Xuhui Huang

Attention Mechanism



© Xuhui Huang

Attention Mechanism



−2	1	0	2
2	3	−1	3

0	−3	−1	−2
2	1	0	−1

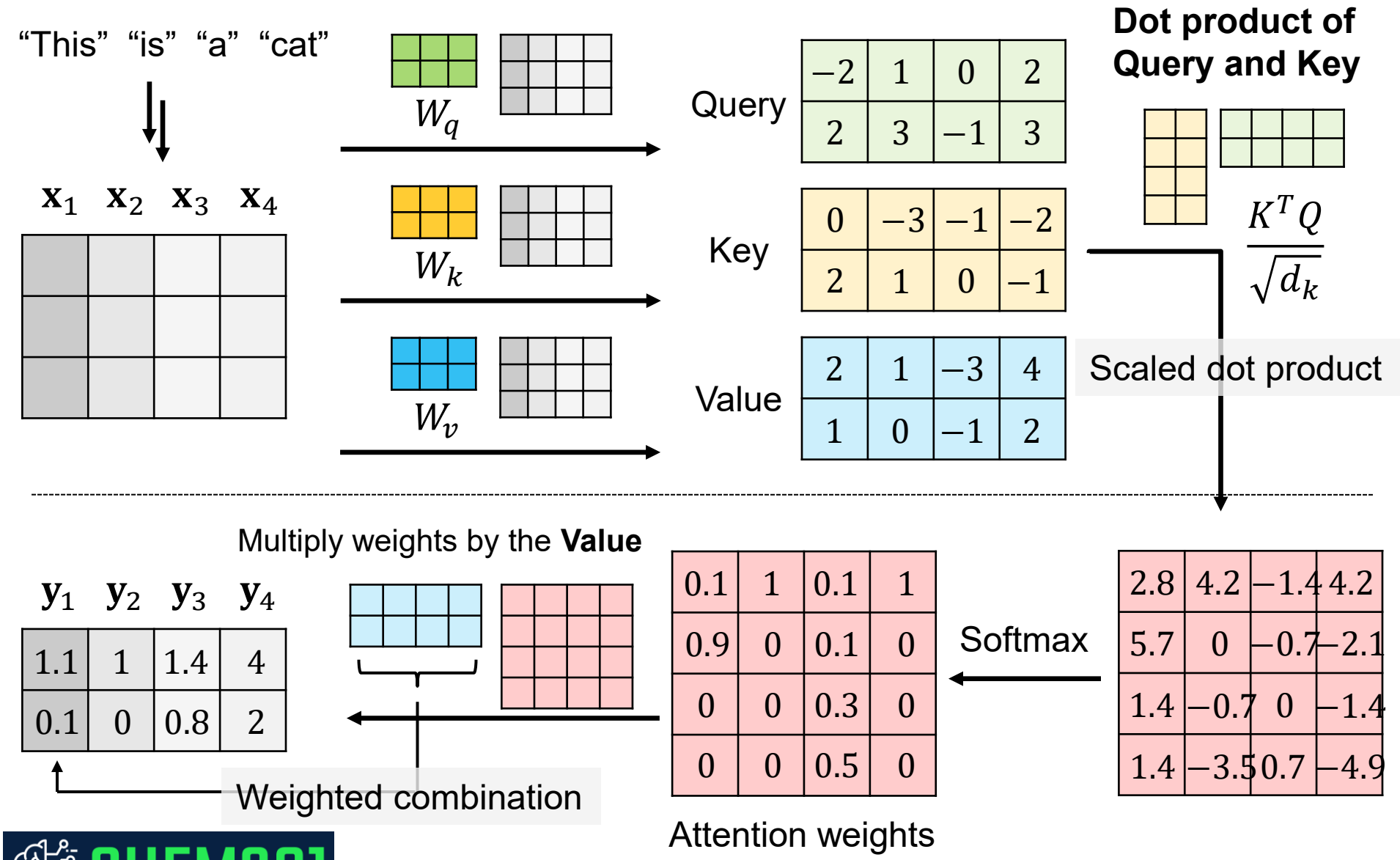
2	1	−3	4
1	0	−1	2

2.8	4.2	−1.4	4.2
5.7	0	−0.7	−2.1
1.4	−0.7	0	−1.4
1.4	−3.5	0.7	−4.9

0.1	1	0.1	1
0.9	0	0.1	0
0	0	0.3	0
0	0	0.5	0

© Xuhui Huang

Attention Mechanism



−2	1	0	2
2	3	−1	3

0	−3	−1	−2
2	1	0	−1

2	1	−3	4
1	0	−1	2

2.8	4.2	−1.4	4.2
5.7	0	−0.7	−2.1
1.4	−0.7	0	−1.4
1.4	−3.5	0.7	−4.9

0.1	1	0.1	1
0.9	0	0.1	0
0	0	0.3	0
0	0	0.5	0

1.1	1	1.4	4
0.1	0	0.8	2

© Xuhui Huang

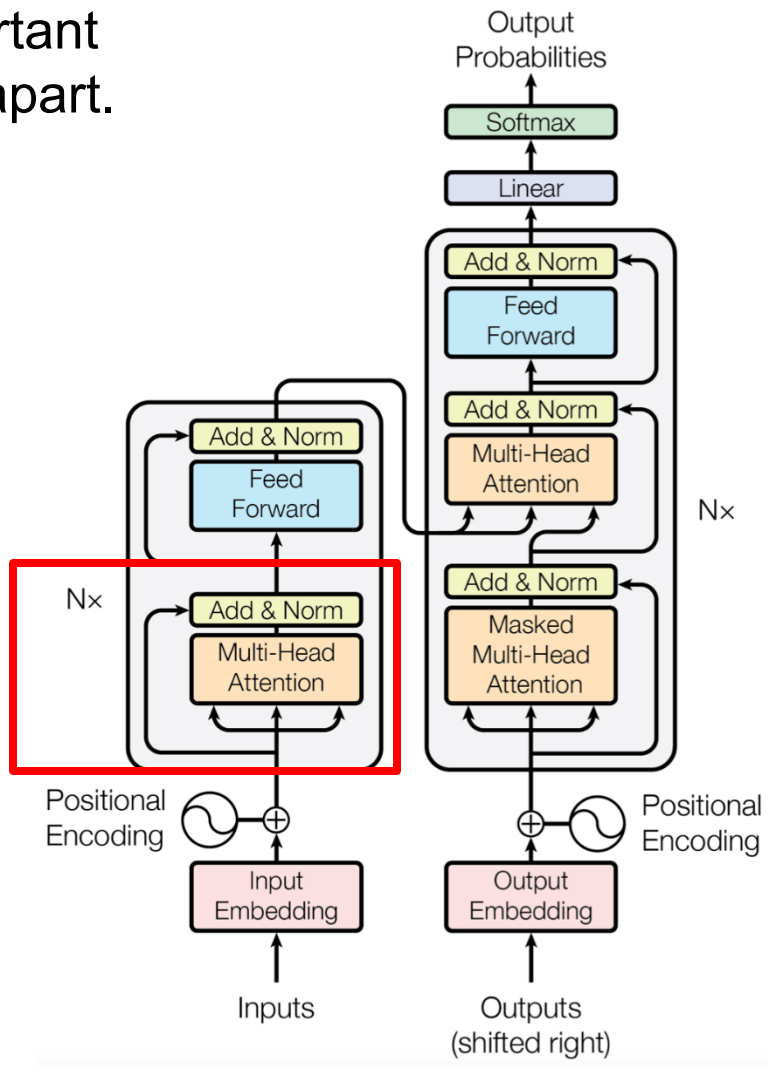
Attention Mechanism

Attention helps the model focus on important words in a sentence, even if they are far apart.

Self-Attention: Each word compares itself to all other words to determine their importance.

Multi-Head Attention: Uses multiple sets of Q, K, and V to capture different types of relationships between words.

Benefit: Allows the model to capture meaning across the entire sentence, rather than relying only on nearby words.

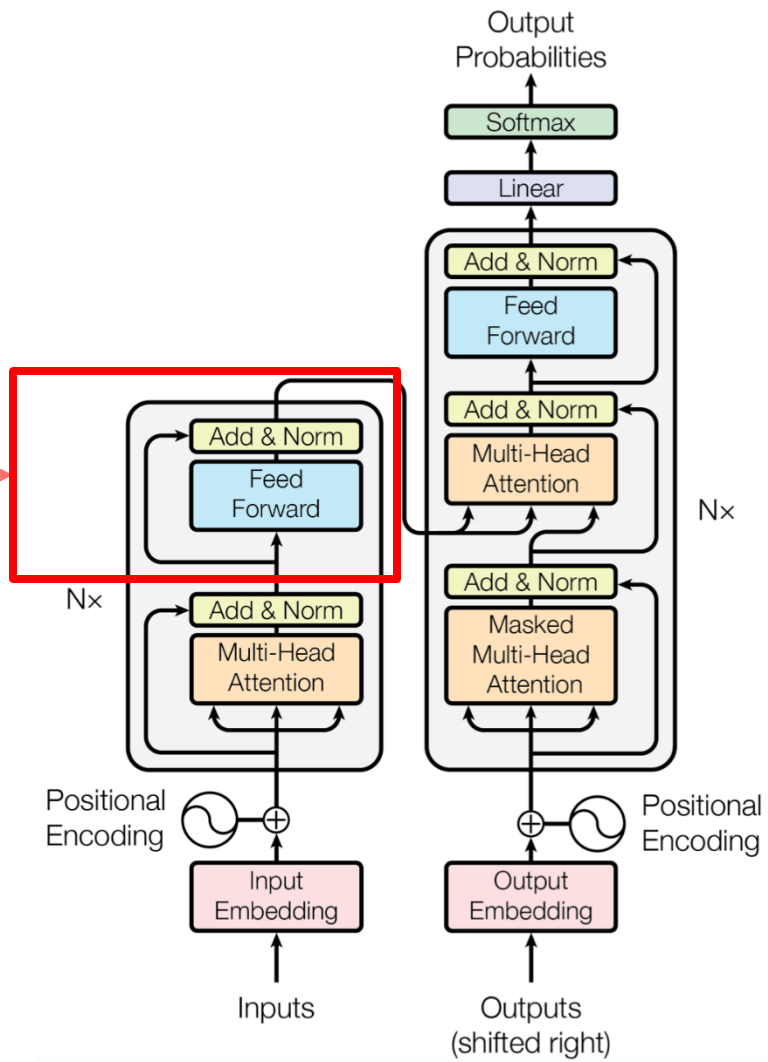


Vaswani, Ashish, et al. "Attention is all you need." *Advances in neural information processing systems* 30 (2017).

© Xuhui Huang

Fully Connected Layers in Transformer

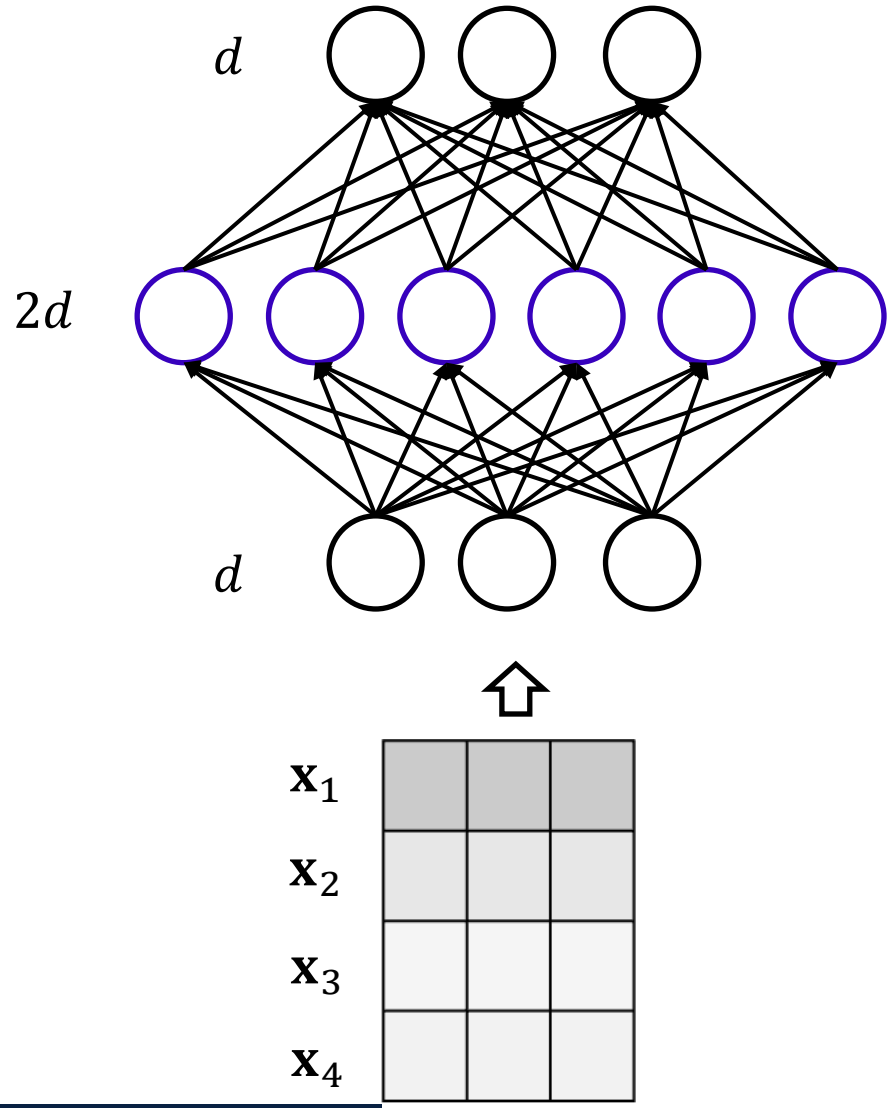
Fully connected layers refine the attention output and helps the model learn more abstract and high-level features.



Vaswani, Ashish, et al. "Attention is all you need." *Advances in neural information processing systems* 30 (2017).

© Xuhui Huang

Fully Connected Layer in Transformer

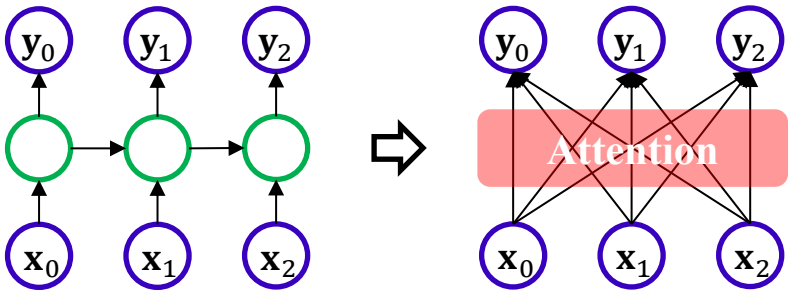


- **First Linear Layer:** Expands the dimension of the input (typically increases by a factor of 2 or 4).
- **Activation (ReLU)**
- **Second Linear Layer:** Projects the expanded dimension back to the original size.

© Xuhui Huang

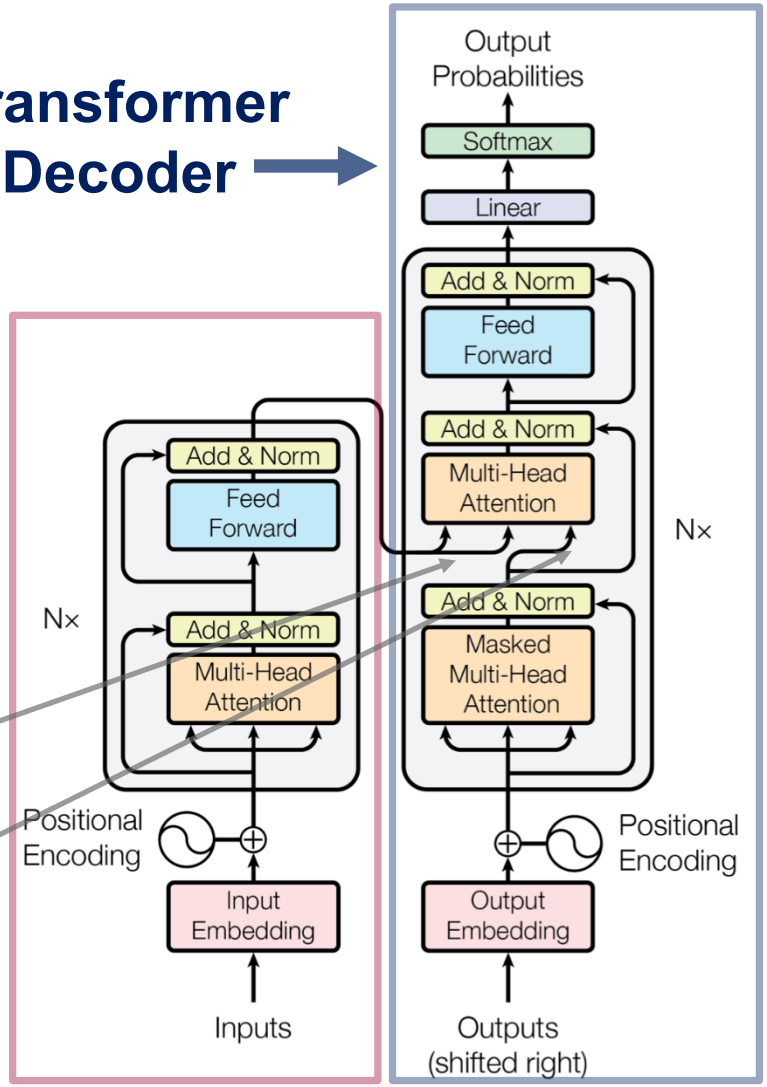
Key Advantages of Transformer

Attention is All You Need!



Transformer Decoder

Transformer Encoder



Query and Keys from Encoder

Values from Decoder

Vaswani, Ashish, et al. "Attention is all you need." *Advances in neural information processing systems* 30 (2017).

© Xuhui Huang

Exercise

What is the main purpose of position encoding in a Transformer?

(A). To prevent overfitting during training

(B). To give information about word order to the model

(C). To reduce the size of the input embeddings

(D). To combine different attention heads

© Xuhui Huang

Exercise

What is the main purpose of position encoding in a Transformer?

(A). To prevent overfitting during training

(B). To give information about word order to the model

(C). To reduce the size of the input embeddings

(D). To combine different attention heads

© Xuhui Huang

Exercise

In self-attention, which component is used to determine the importance of words?

(A). Query (Q)

(B). Key (K)

(C). Value (V)

(D). Embedding

© Xuhui Huang

Exercise

In self-attention, which component is used to determine the importance of words?

(A). Query (Q)

(B). Key (K)

(C). Value (V)

(D). Embedding

Query and Key together determine the attention weights. Without them, the model wouldn't know which relationships to prioritize.

© Xuhui Huang

Exercise

In multi-head attention, how are the outputs of different attention heads combined?

(A). They are averaged.

(B). They are concatenated and passed through a linear layer.

(C). The highest scoring head is chosen.

(D). They are passed individually through the feedforward layer.

© Xuhui Huang

Exercise

In multi-head attention, how are the outputs of different attention heads combined?

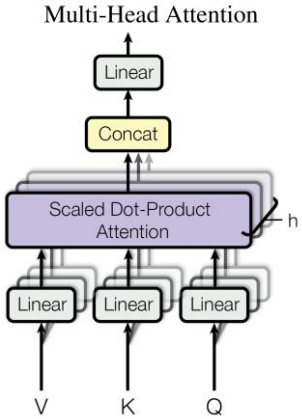
(A). They are averaged.

(B). They are concatenated and passed through a linear layer.

(C). The highest scoring head is chosen.

(D). They are passed individually through the feedforward layer.

Concatenate along the embedding dimension to combine information from different perspectives, and pass through a linear layer to transform the concatenated output back to the original embedding dimension.



© Xuhui Huang

Exercise

What role does softmax play in the attention mechanism?

(A). It normalizes the attention scores into probabilities.

(B). It reduces the input size.

(C). It introduces non-linearity between layers.

(D). It combines positional encodings with embeddings.

© Xuhui Huang

Exercise

What role does softmax play in the attention mechanism?

(A). It normalizes the attention scores into probabilities.

(B). It reduces the input size.

(C). It introduces non-linearity between layers.

(D). It combines positional encodings with embeddings.

© Xuhui Huang

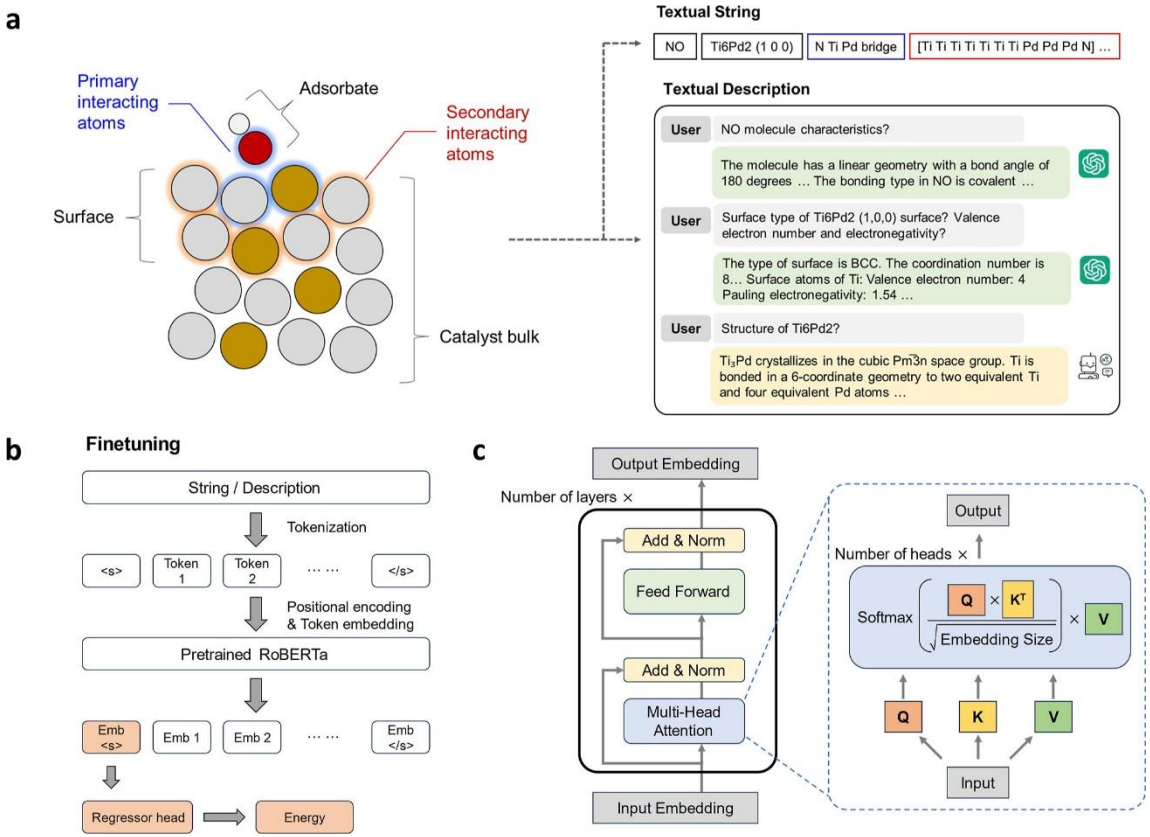
Outline

- Introduction to Large Language Models (LLMs)
- Transformer
- **LLMs in Chemistry and Biology**
- Applying a pre-trained Protein LLM to predict GPCR agonists – Hands-on implementation for a real chemistry problem.

© Xuhui Huang

Large Language Models in Chemistry

CatBERTa finetuned the pretrained large language model RoBERTa on OpenCatalyst2020 (OC20) data set to accurately predict adsorption energy of catalyst



Model size: 355 Million parameters

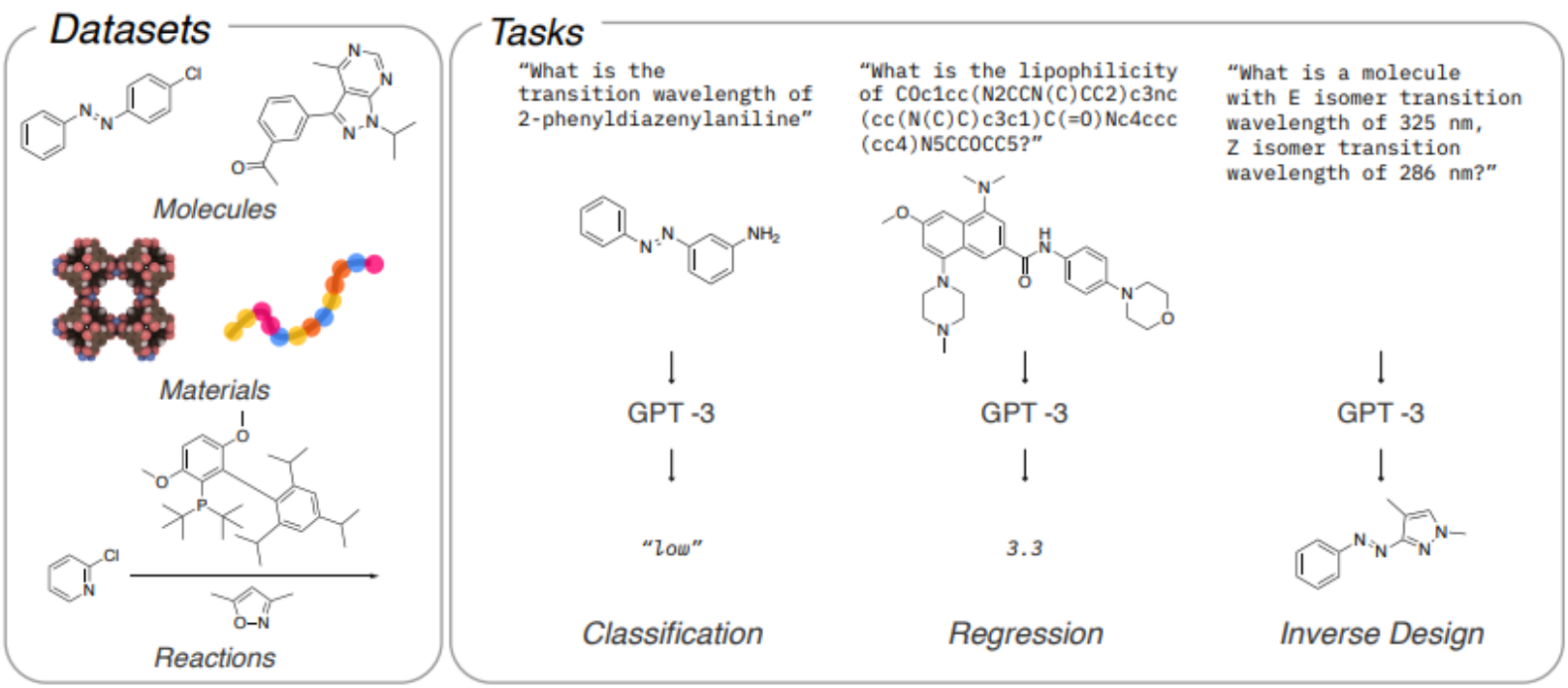
RoBERTa: a robustly optimized variant of the Bidirectional Encoder Representations from Transformers (BERT) model, which is an encoder-only model.

ACS Catal., 13, 24, 16032-16044, (2023)

© Xuhui Huang

Large Language Model in Chemistry

GPTChem finetuned the pretrained large language model GPT-3 on multiple regression and classification tasks for the purpose of property prediction and inverse design.



Model size: 175 Billion parameters

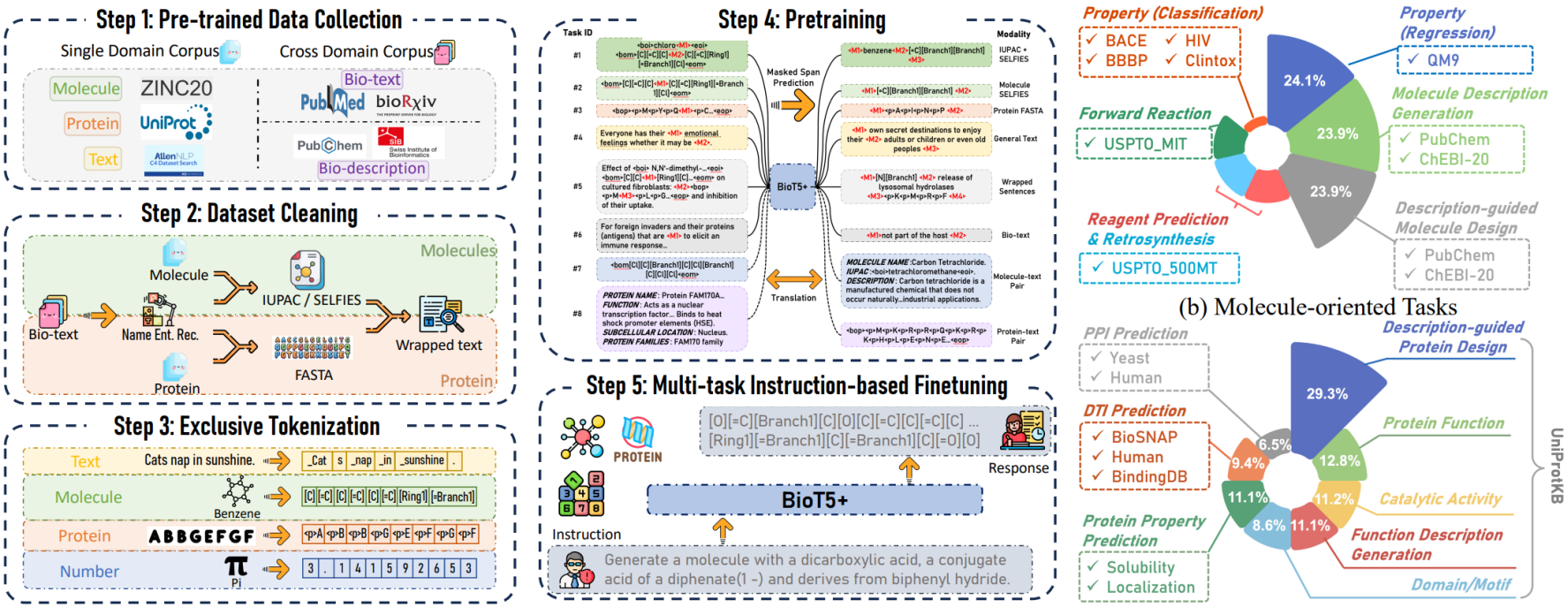
GPT: Generative Pre-trained Transformer which is a decoder-only model.

Jablonka KM, et al. ***ChemRxiv***. doi:10.26434/chemrxiv-2023-fw8n4-v3

© Xuhui Huang

Large Language Models in Chemistry

BioT5+ pretrained and finetuned T5 model on Bio-text and molecule data for multi-tasks: retrosynthesis, molecule description generation, drug-target interaction, etc.



(a) Overview of BioT5+

(b) Molecule-oriented Tasks

(c) Protein-oriented Tasks

Model size: 252 Million parameters

T5: Text-to-Text Transfer Transformer, an encoder-decoder model

Pei, Q., et al. ***arXiv***:*2402.17810*.

© Xuhui Huang

Outline

- Introduction to Large Language Models (LLMs)
- Transformer
- LLMs in Chemistry and Biology
- **Applying a pre-trained Protein LLM to predict GPCR agonists – Hands-on implementation for a real chemistry problem.**

© Xuhui Huang

nature chemistry

Article

https://doi.org/10.1038/s41557-024-01532-x

Machine learning designs new GCGR/GLP-1R dual agonists with enhanced biological potency

Received: 26 December 2021

Accepted: 8 April 2024

Published online: 16 May 2024

Check for updates

Anna M. Puszkarska[1,6], Bruck Taddese[2,7], Jefferson Revell[2], Graeme Davies[3], Joss Field[3], David C. Hornigold[3], Andrew Buchanan[4], Tristan J. Vaughan[4,8] & Lucy J. Colwell[1,5]

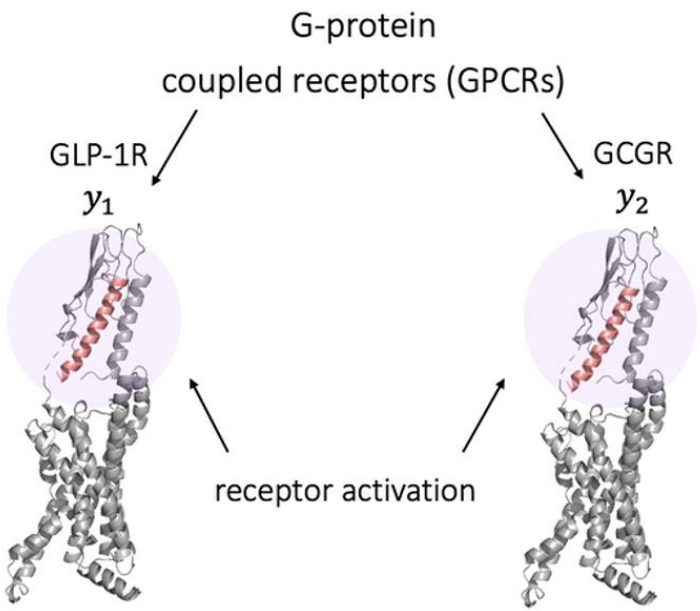
Several peptide dual agonists of the human glucagon receptor (GCGR) and the glucagon-like peptide-1 receptor (GLP-1R) are in development for the treatment of type 2 diabetes, obesity and their associated complications. Candidates must have high potency at both receptors, but it is unclear whether the limited experimental data available can be used to train models that accurately predict the activity at both receptors of new peptide variants. Here we use peptide sequence data labelled with in vitro potency at human GCGR and GLP-1R to train several models, including a deep multi-task neural-network model using multiple loss optimization. Model-guided sequence optimization was used to design three groups of peptide variants, with distinct ranges of predicted dual activity. We found that three of the model-designed sequences are potent dual agonists with superior biological activity. With our designs we were able to achieve up to sevenfold potency improvement at both receptors simultaneously compared to the best dual-agonist in the training set.

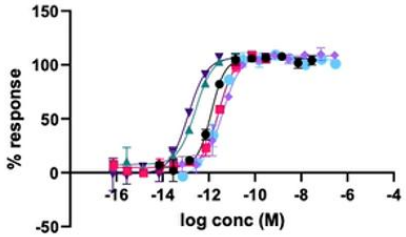
Use a pretrained ESM2 model to predict dataset in this paper

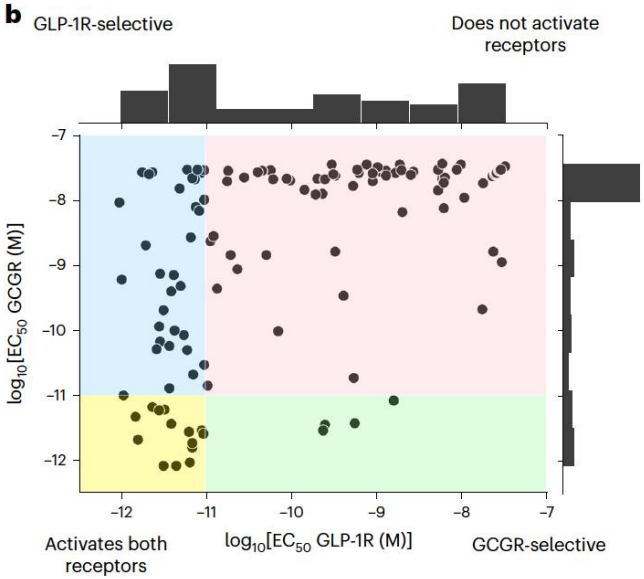
© Xuhui Huang

Designing Dual Agonists of two Human GPCRs

Training data: 125 experimentally characterized peptides (31 a.a. max) and their EC50 against two GPCRs (GCGR and GLP-1R).







Nature Chemistry, **16**, 1436–1444, (2024)

© Xuhui Huang

Featurization

Featurization: Tokenization of amino acids in ESM2

Vocabulary: '<cls>': 0, '<pad>': 1, '<eos>': 2, '<unk>': 3, 'L': 4, 'A': 5, 'G': 6, 'V': 7, 'S': 8, 'E': 9, 'R': 10, 'T': 11, 'I': 12, 'D': 13, 'P': 14, 'K': 15, 'Q': 16, 'N': 17, 'F': 18, 'Y': 19, 'M': 20, 'H': 21, 'W': 22, 'C': 23, 'X': 24, 'B': 25, 'U': 26, 'Z': 27, 'O': 28, '.': 29, '-': 30, '<null_1>': 31, '<mask>': 32

The size of vocabulary used in ESM2 is **33**.

- **'<cls>'** is appended to the start of the sequence for downstream classification/regression tasks.
- **'<eos>'** is appended at the end of the sequence.

Reference_ch8_PLM_colab.ipynb

© Xuhui Huang

Featurization

Featurization: Tokenization of amino acids in ESM2

Vocabulary: '<cls>': 0, '<pad>': 1, '<eos>': 2, '<unk>': 3, 'L': 4, 'A': 5, 'G': 6, 'V': 7, 'S': 8, 'E': 9, 'R': 10, 'T': 11, 'I': 12, 'D': 13, 'P': 14, 'K': 15, 'Q': 16, 'N': 17, 'F': 18, 'Y': 19, 'M': 20, 'H': 21, 'W': 22, 'C': 23, 'X': 24, 'B': 25, 'U': 26, 'Z': 27, 'O': 28, '.': 29, '-': 30, '<null_1>': 31, '<mask>': 32

Example:

- **Sequence:** HSQGTFTSDYSKYLDSRRAQDFVQWLEEGE
- **Tokens:** *['H', 'S', 'Q', 'G', 'T', 'F', 'T', 'S', 'D', 'Y', 'S', 'K', 'Y', 'L', 'D', 'S', 'R', 'R', 'A', 'Q', 'D', 'F', 'V', 'Q', 'W', 'L', 'E', 'E', 'G', 'E']*
- Tokenizer returns two items:

  'input_ids': *tensor([[0, 21, 8, 16, 6, 11, 18, 11, 8, 13, 19, 8, 15, 19, 4, 13, 8, 10, 10, 5, 16, 13, 18, 7, 16, 22, 4, 9, 9, 6, 9, 2]])*.

  'attention_mask': *tensor([[1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1]])*.

  Attention mask can be used to mask out padding tokens. Here since the peptide length is fixed, we won't use this item.

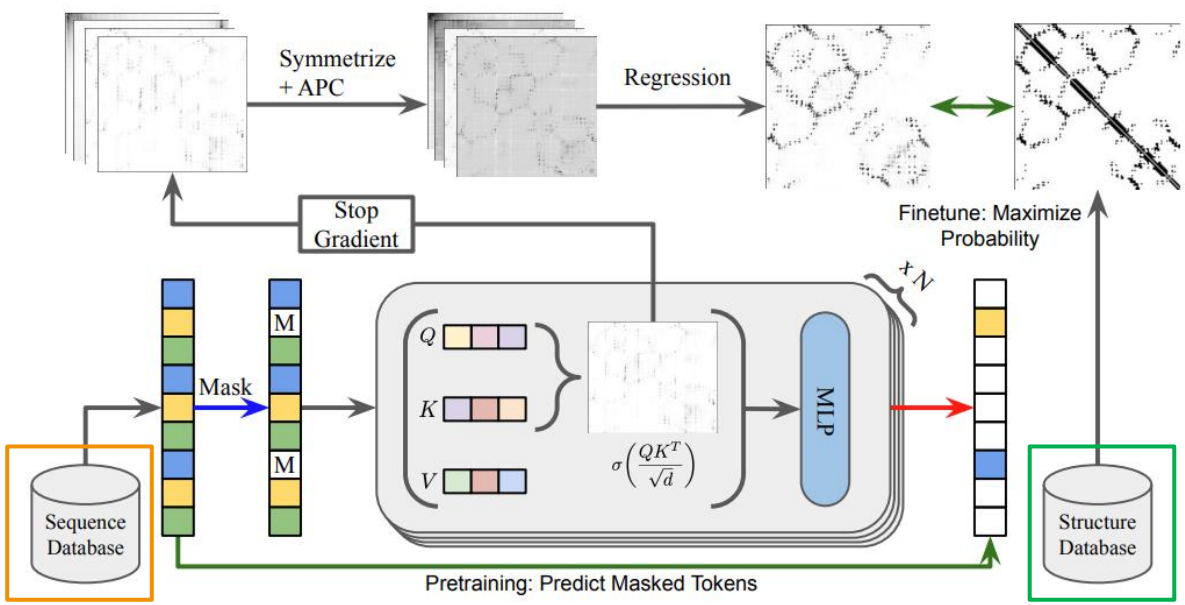
Reference_ch8_PLM_colab.ipynb

© Xuhui Huang

Large Protein Language Models

ESM-1b (Evolutionary Scale Modeling) is a pretrained self-supervised masked language model on the UniRef50 dataset. It learns context-based embeddings of each residue in the input sequence.

The attention maps of ESM-1b are extracted to further train on a small number of proteins to determine the importance of attention heads, used for contact map prediction.



Pretrained on sequence data

Downstream task trained on structure data for contact map prediction

Roshan Rao, et al. ***BioRxiv**:2020.12.15.422761*

© Xuhui Huang

Large Protein Language Models

ESM2 used Rotary Position Embedding (RoPE) in attention blocks to better capture the position information of residues.

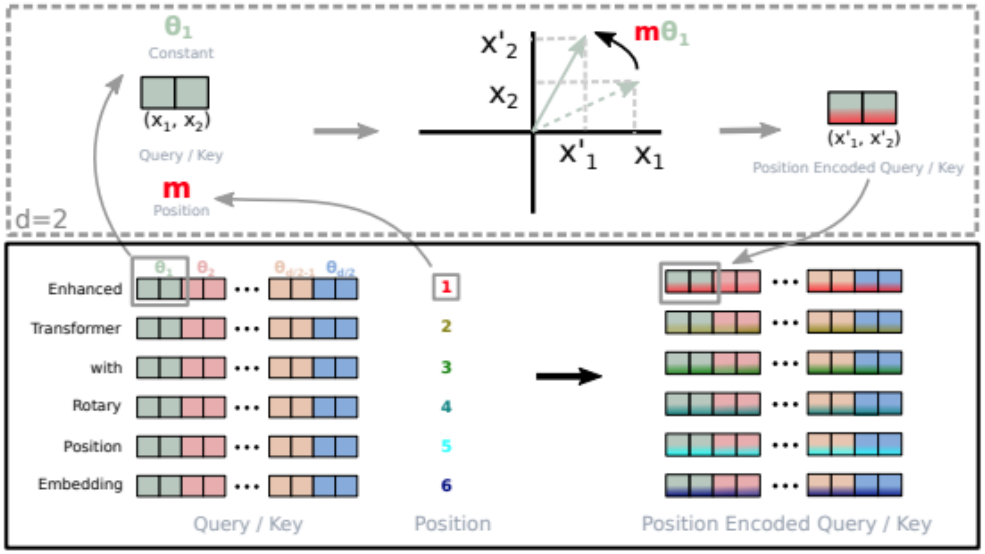


Figure 1: Implementation of Rotary Position Embedding(RoPE).

$$f_{\{q,k\}}(\boldsymbol{x}_m, m) = \boldsymbol{R}^d_{\Theta,m}\boldsymbol{W}_{\{q,k\}}\boldsymbol{x}_m$$

$$\boldsymbol{R}^d_{\Theta,m} = \begin{pmatrix} \cos m\theta_1 & -\sin m\theta_1 & 0 & 0 & \cdots & 0 & 0 \\ \sin m\theta_1 & \cos m\theta_1 & 0 & 0 & \cdots & 0 & 0 \\ 0 & 0 & \cos m\theta_2 & -\sin m\theta_2 & \cdots & 0 & 0 \\ 0 & 0 & \sin m\theta_2 & \cos m\theta_2 & \cdots & 0 & 0 \\ \vdots & \vdots & \vdots & \vdots & \ddots & \vdots & \vdots \\ 0 & 0 & 0 & 0 & \cdots & \cos m\theta_{d/2} & -\sin m\theta_{d/2} \\ 0 & 0 & 0 & 0 & \cdots & \sin m\theta_{d/2} & \cos m\theta_{d/2} \end{pmatrix}$$

RoPE encodes position information by rotating the query and key matrices.

Jianlin Su, et al. ***arXiv***:*2104.09864*

Zeming Lin, et al. ***BioRxiv***:*2022.07.20.500902*

© Xuhui Huang

ESM-2 Architecture Example

If you go to huggingface or facebook/esm for pretrained models, the models have names in the format of “esm2_t6_8M_UR50D”, which means that it is a transformer architecture with 6 layers, containing ~8 million parameters and trained on the UR50D dataset. UR50 means “UniRef50”. You can choose the proper model size according with your hardware constraints.

Token Embeddings

Six encoder blocks, with each containing:

- attention block with RoPE
- Intermediate block
- LayerNorm

Each residue has a 320-dim context-based embedding

Pooling layer:

Each sequence is pooled as a 320-dim vector for downstream tasks

Contact head:

used for residue contact map predictions

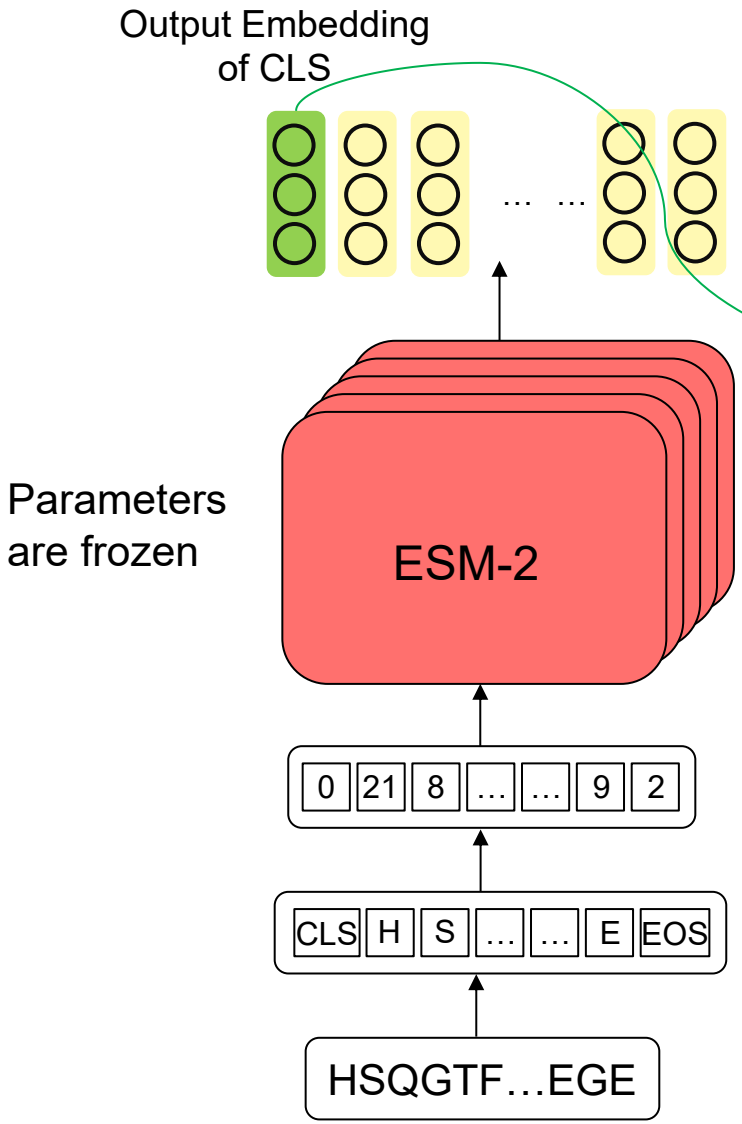
esm2_t6_8M_UR50D

```
EsmModel(
  (embeddings): EsmEmbeddings(
    (word_embeddings): Embedding(33, 320, padding_idx=1)
    (dropout): Dropout(p=0.0, inplace=False)
    (position_embeddings): Embedding(1026, 320, padding_idx=1)
  )
  (encoder): EsmEncoder(
    (layer): ModuleList(
      (0-5): 6 x EsmLayer(
        (attention): EsmAttention(
          (self): EsmSelfAttention(
            (query): Linear(in_features=320, out_features=320, bias=True)
            (key): Linear(in_features=320, out_features=320, bias=True)
            (value): Linear(in_features=320, out_features=320, bias=True)
            (dropout): Dropout(p=0.0, inplace=False)
            (rotary_embeddings): RotaryEmbedding()
          )
          (output): EsmSelfOutput(
            (dense): Linear(in_features=320, out_features=320, bias=True)
            (dropout): Dropout(p=0.0, inplace=False)
          )
          (LayerNorm): LayerNorm((320,), eps=1e-05, elementwise_affine=True)
        )
        (intermediate): EsmIntermediate(
          (dense): Linear(in_features=320, out_features=1280, bias=True)
        )
        (output): EsmOutput(
          (dense): Linear(in_features=1280, out_features=320, bias=True)
          (dropout): Dropout(p=0.0, inplace=False)
        )
        (LayerNorm): LayerNorm((320,), eps=1e-05, elementwise_affine=True)
      )
    )
    (emb_layer_norm_after): LayerNorm((320,), eps=1e-05, elementwise_affine=True)
  )
  (pooler): EsmPooler(
    (dense): Linear(in_features=320, out_features=320, bias=True)
    (activation): Tanh()
  )
  (contact_head): EsmContactPredictionHead(
    (regression): Linear(in_features=120, out_features=1, bias=True)
    (activation): Sigmoid()
  )
)
```

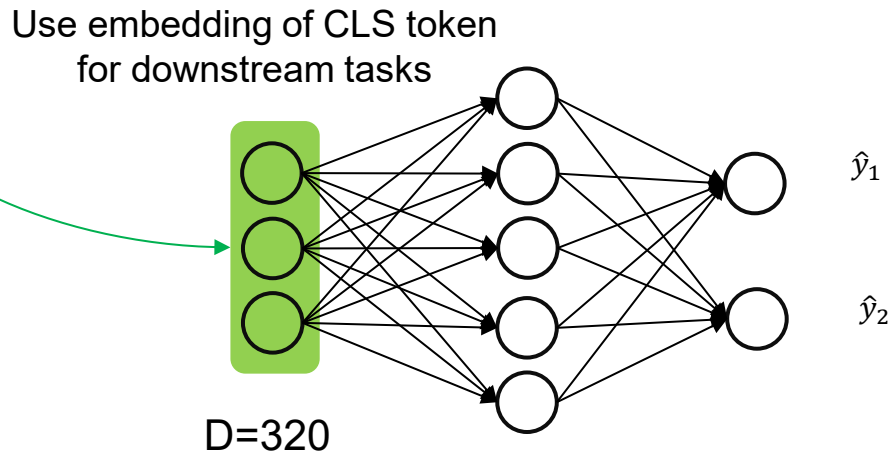
RoPE

© Xuhui Huang

Downstream Task Setup



Reference_ch8_PLM_colab.ipynb



Trainable multi-layer perceptron (MLP) on top of the ESM2 model

Number of all params: 7860795

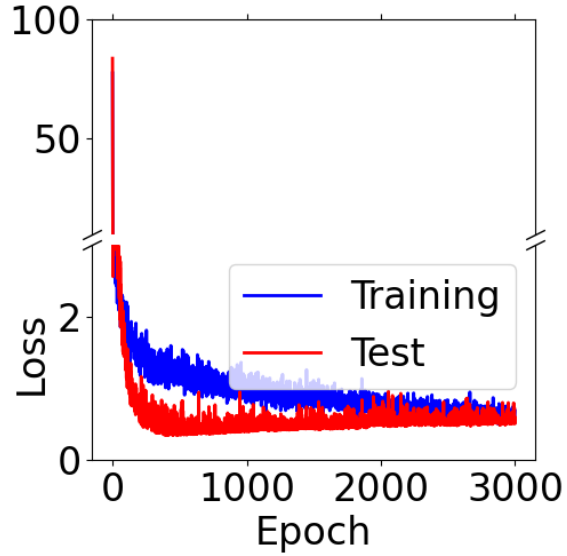
Number of trainable params: 20674

Trainable ratio: **0.263%**

For comparison, number of trainable params in **Reference_ch6_Part_1_CNNs_colab.ipynb** is **649026**.

© Xuhui Huang

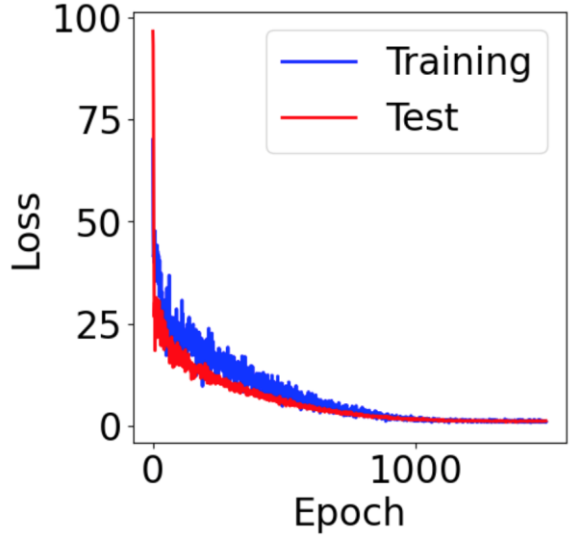
Downstream Training



Reference_ch8_PLM_colab.ipynb

Loss function: Since this is a multi-task regression model, both tasks contribute equally to the total loss.

$$\ell(f_\theta) = 0.5MSE(GCGR) + 0.5\ MSE\ (GLP-1R)$$

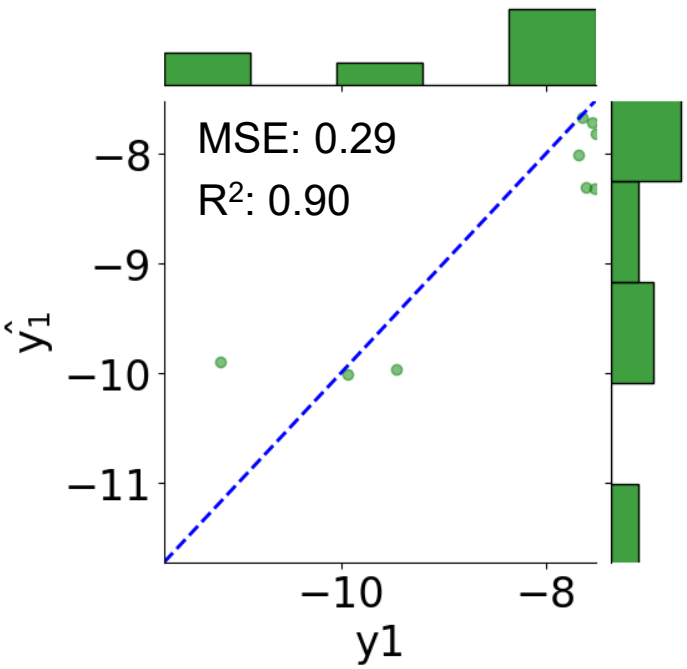


For comparison, training curve from Reference_ch6_Part1_CNN_colab.ipynb

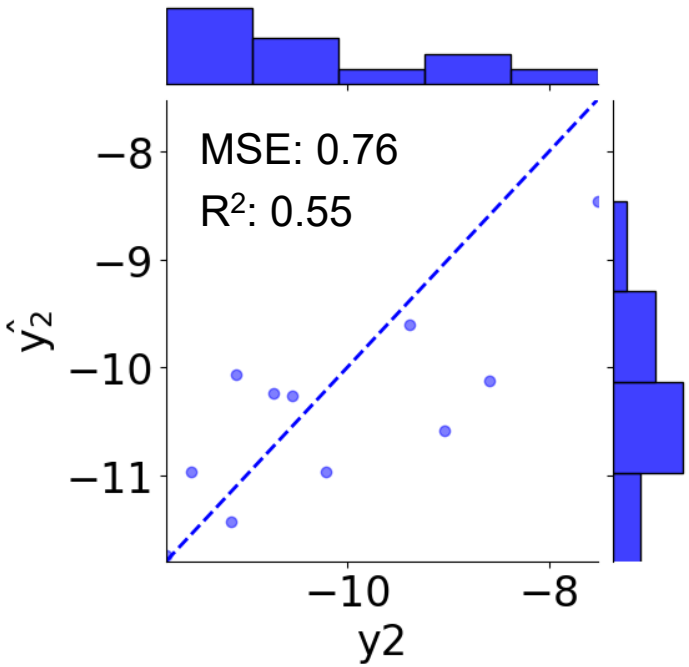
© Xuhui Huang

Evaluation

GCGR



GLP-1R



Nature Chemistry, **16**, 1436–1444, (2024)

© Xuhui Huang